

第3讲 Python 程序的控制结构

在本章中，我们将会介绍 Python 中的程序控制结构。计算机编程语言的研究表明，使用顺序、分支、循环三种程序控制结构即可表达任意的可计算函数，该理论被成为“结构化程序理论”。顺序结构比较容易理解，即代码语句逐行运行，因此，本章的主要介绍分支和循环结构。

3.1 分支结构：if 语句

分支结构可以简单分为单分支结构和二分支结构，而通过二分支结构的嵌套和组合，我们则可以得到多分支结构。

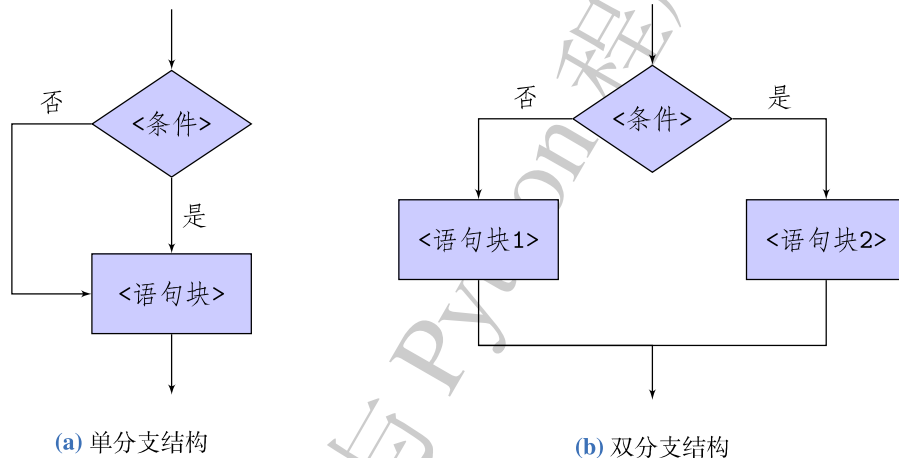


图 3.1: 分支控制结构

Python 中的分支结构主要是通过 if 语句实现的，单分支结构的基本语法为：

```
1 if <条件>:
2     <语句块>
```

双分支结构的基本语法为：

```
1 if <条件>:
2     <语句块1>
3 else:
4     <语句块2>
```

其中<条件>部分应为一个能够判断真假的语句，它可以是一个简单的比较运算符，也可以是一个稍复杂的逻辑表达式，还可以是一个返回布尔变量的函数。例如：

```
1 pm = 75
2 if 0<= pm < 35:
3     print('空气质量好')
4 else:
5     print('空气质量不好')
```

输出结果为

```
1 空气质量不好
```

Python 中的双分支结构还有一种更为紧凑的写法:

```
1 >>> pm = 75
2 >>> print('空气质量好') if 0<= pm < 35 else print('空气质量不好')
3 空气质量不好
```

或者:

```
1 >>> pm = 75
2 >>> output = '空气质量好' if 0<= pm < 35 else '空气质量不好'
3 >>> print(output)
4 空气质量不好
```

多分支结构可以使用双分支结构的嵌套来实现, 为了方便程序编写, Python 提供了一个`elif`的保留字, 作为`else if`的缩写。

```
1 pm = 75
2 if 0 <= pm < 35:
3     print('空气优质, 快去户外运动')
4 elif 35 <= pm < 75:
5     print('空气良好, 适度户外活动')
6 else:
7     print('空气污染, 请小心')
```

上述代码的输出为:

```
1 空气污染, 请小心
```

3.2 循环结构: for语句和while语句

循环结构是指根据条件判断结果来反复执行代码的过程, 常用于遍历某些结构中的元素。

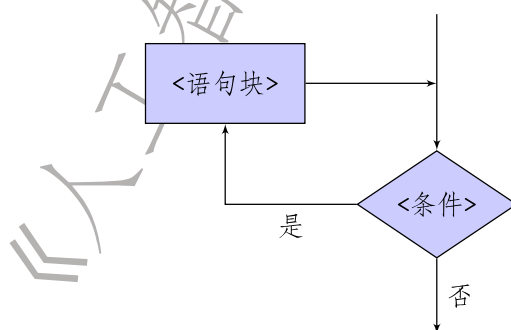


图 3.2: 循环结构

Python 中实现循环结构一般采用`for`和`while`来实现。使用`for`来实现循环结构的语法为:

```
1 for <循环变量> in <遍历结构>:
2     <语句块>
```

该语句表示, 每次循环时, 从<遍历结构>中取出一个元素, 用<循环变量>来表示该元素, 并执行<语句块>。其中, <遍历结构>可以是一个字符串、列表、生成器等。例如, 使用`range()`函数可以生成 n 个连续整数, 可以将一段代码执行 n 次:

```
1 for i in range(3):  
2     print(i)
```

上述代码的输出为:

```
1 0  
2 1  
3 2
```

值得注意的是, `range()`函数生成的连续 n 个整数是从 0 开始, 到 $n-1$ 结束, 而不包含 n 。

我们也可以使用`for`循环语句来遍历一个字符串, 此时, <遍历结构>为一个字符串, <循环变量>为字符串中的一个字符。

```
1 for s in 'Python':  
2     print(s)
```

上述代码的输出为:

```
1 P  
2 y  
3 t  
4 h  
5 o  
6 n
```

我们还可以使用`for`循环来遍历一个文本文件, 此时, <遍历结构>为一个文件, <循环变量>为该文件中的某一行文本。

```
1 with open('text_file.txt') as text_file:  
2     for line in text_file:  
3         print(line.strip())
```

上述代码将遍历文本文件'`text_file.txt`'的每一行, 并将每一行的内容输出到屏幕上。

使用`while`语句实现循环的语法为:

```
1 while <条件>:  
2     <语句块>
```

上述代码的含义为, 只要<条件>成立, 就执行<语句块>, 因此, 使用`while`时, 一般并不需要我们事先知道循环执行的次数。

```
1 count = 0  
2 while count < 3:  
3     print('Programming is fun')  
4     count += 1
```

上述代码的执行结果为:

```
1 Programming is fun  
2 Programming is fun  
3 Programming is fun
```

上面的例子中，`while`语句块中`count += 1`的作用是改变当前状态，从而改变<条件>的判断，如果语句块中没有改变<条件>判断的语句，那么该`while`循环将永远执行，成为一个死循环。

下面的代码使用二分查找法来近似 $\sqrt{2}$ 的值：

```
1 lower_bound = 1      # 下界
2 upper_bound = 2      # 上界
3 eps = 1e-8          # 精度
4 while upper_bound - lower_bound > eps:
5     mid = (lower_bound + upper_bound) * 0.5
6     if mid ** 2 > 2:
7         upper_bound = mid    # 更新上界
8     else:
9         lower_bound = mid    # 更新下界
10 print(lower_bound)
```

上述代码将输出

```
1 1.4142135605216026
```

在使用`for`或`while`来循环执行代码时，我们还可以使用`continue`和`break`语句来对循环执行过程进行更为细致的控制。`continue`语句的作用是跳过当前循环，直接进入下一次循环执行前的条件判断，而`break`语句的作用是直接跳出整个循环，即提前结束整个循环。举例如下：

```
1 for s in 'Python':
2     if s == 't':
3         continue
4     print(s, end='')
```

上述代码执行后的输出为：

```
1 Pyhon
```

而如果将上述代码中的`continue`改为`break`，即改为下面的代码

```
1 for s in 'Python':
2     if s == 't':
3         break
4     print(s, end='')
```

那么其执行结果为

```
1 Py
```

产生此种区别的原因就在于，`continue`仅仅会跳出当前循环的执行，所以't'之后的字母仍然会输出，而`break`则会退出整个循环，因此't'之后的字母就不会输出了。

3.3 异常处理

在编程中经常会出现程序运行异常的情况，例如，需要打开读取的文件不存在，或者用户输入的内容不合法等。出现这些情况时，我们就需要对这些异常进行处理，否则，程序就会崩溃。Python 中的异常处理一般使用`try ... except ...`语句来实现，具体的语法为：

```

1 try:
2     <语句块1>
3 except <异常类型>:    <语句块2>

```

Python 解释器会尝试执行<语句块1>，而except语句则会尝试捕捉<语句块1>执行过程中出现的错误，如果错误的类型为<异常类型>，那么 Python 解释器就会执行<语句块2>。我们也可以增加except的部分，从而对不同的错误类型进行针对性处理，即：

```

1 try:
2     <语句块1>
3 except <异常类型1>:
4     <语句块2>
5 except <异常类型2>:
6     <语句块3>

```

我们也可以忽略<异常类型>部分，从而让except捕捉所有的错误。例如：

```

1 num_str = input('请输入一个数：\n')
2 try:
3     num = float(num_str)
4     print('{}的平方是{}'.format(num, num ** 2))
5 except:
6     print('输入不是一个数')

```

上面的例子中，我们提示用户输入一个数，然后我们尝试将用户的输入转换为一个浮点数，如果转换过程中出现了错误，这就说明用户的输入并不是一个数，此时的错误会被except捕捉到，并且输出错误的说明'输入不是一个数'。用户输入为“2.5”时，上述代码的执行结果为：

```

1 请输入一个数：
2 2.5
3 2.5的平方是6.25

```

而如果用户输入“qwer”，那么代码的执行结果为：

```

1 请输入一个数：
2 qwer
3 输入不是一个数

```

此外，Python 中的异常处理还支持else和finally关键字来对异常处理过程进行更精细的控制。其语法结构为：

```

1 try:
2     <语句块1>    # 尝试执行的语句块
3 except:
4     <语句块2>    # 出现异常时执行的语句块
5 else:
6     <语句块3>    # 出现异常时执行的语句块
7 finally:
8     <语句块4>    # 一定会执行的语句块

```

3.4 函数的定义与调用

在编程中，我们有时需要相似场合执行相似的代码，为了方便使用，我们可以定义一个“函数”，将上述代码封装起来，形成一个完整的功能，在需要执行类似代码的场合直接调用该函数即可实现该功能。封装成函数后，不仅可以使功能的实现变得简单，也使得后期的维护变得方便，因为我们只需修改函数的定义，所有该函数的调用就都得到修改，因此，函数也可以看作是一种“子程序”。另外，函数还提供参数输入功能，以实现对不同数据的处理。

3.4.1 函数的定义与调用

Python 中内置了不少较为底层的函数，例如`print()`、`input()`等，Python 标准库中也包含一些函数，例如`math`模块中的`log()`等。当然，我们也可以自己定义函数，在 Python 中，定义一个函数通常需要指定一个函数名，输入参数，有时还需要给出函数的返回值，用以将数据处理的结果反馈给调用者。Python 中自定义函数的基本框架如下：

```
1 def <函数名>(<参数列表>):  
2     <函数体>  
3     return <返回值列表>
```

在上面的框架中，<参数列表>和<返回值列表>都可以为空，甚至`return`语句都可以省略，这需要根据实际情况来决定。例如：

```
1 def print_happy():  
2     print('Happy birthday to you!')
```

上面的函数没有输入参数，也没有返回值，该函数的作用是在屏幕上打印字符串'Happy birthday to you!'，而下面的函数则可以根据不同人的名字输出不同的字符串：

```
1 def print_happy_with_name(name):  
2     print('Happy birthday to you, {}'.format(name))
```

该函数有一个输入参数`name`，该参数在函数体中将被作为输出字符串的一部分，从而实现根据不同输入来实现不同输出。我们一般将函数定义中的参数叫做“形式参数”，简称“形参”，而将调用函数时实际传入函数的参数称为“实际参数”，简称“实参”。

需要使用某函数时，我们可以通过该函数的函数名来进行调用，并指定函数的参数：<函数名>(<参数列表>)。指定函数参数时，要让实参与形参一一对应。函数也可以嵌套调用，即在函数中调用另一个函数，例如，下面的函数将调用上面两个函数，并输出生日歌的歌词：

```
1 def print_lyrics(name):  
2     print_happy()  
3     print_happy()  
4     print_happy_with_name(name)  
5     print_happy()
```

在该函数中，我们调用了之前定义的两个函数，其中的参数`name`又被当作子函数`print_happy_with_name()`的参数，被子函数输出。若采用'Mike'作为函数`print_lyrics()`的参数，则其输出为：

```
1 Happy birthday to you!  
2 Happy birthday to you!  
3 Happy birthday to you, Mike!
```

```
4 Happy birthday to you!
```

在上面的例子中, `print_lyrics()` 函数在被调用时, 首先调用子函数 `print_happy()`, 进入子函数后, 该子函数执行 `print('Happy birthday to you!')`, 打印第一行歌词到屏幕上, 待函数 `print_happy()` 执行完成, 我们退出该函数, 又返回到函数 `print_lyrics()` 中, 开始执行该函数的后续代码。按照上述的执行过程, `print_lyrics()` 函数不断调用其他函数, 最终输出完整的 4 句歌词。

一般情况下, 传入实参应与函数定义中的实参的顺序相同, 但 Python 还提供另一种参数传递方式, 即按照按参数名传入实参的方法, 其语法为 `<形参名>=<实参名>`, 例如:

```
1 >>> def f(a, b):
2 ...     print('形参a对应的实参为{}, 形参b对应的实参为{}'.format(a, b))
3 ...
4 >>> f(1, 2)
5 形参a对应的实参为1, 形参b对应的实参为2
6 >>> f(b=1, a=2)
7 形参a对应的实参为2, 形参b对应的实参为1
```

在上述代码中, 第 4 行调用函数 `f` 时是按照顺序来传递参数的, 因此第一个实参 1 被传递给第一个形参 `a`。而第 6 行则是按照参数名的方式传入参数的, 因此传递给 `a` 的实参则变为了 2。

除了上述函数定义方法外, Python 还提供一个 `lambda` 保留字, 我们可以利用该保留字定义匿名函数 (也称 `lambda` 函数), 定义匿名函数的语法结构为:

```
1 lambda <参数列表>:<表达式>
```

如果我们需要显式调用该函数, 我们也可以将定义的函数赋值给一个变量, 此时, 该变量就变成了匿名函数的函数名, 相当于我们给匿名函数起了名字。

```
1 <函数名> = lambda <参数列表>:<表达式>
```

上面的匿名函数定义等价于下面的函数定义形式:

```
1 def <函数名>(<参数列表>):
2     return <表达式>
```

与正常定义的函数相比, 匿名函数没有函数体, 因此一般也只用于定义非常简单、一个表达式就可以定义清楚的函数。并且, 相较于正常的函数定义, 匿名函数显得不太“正式”, 所以, 我们一般也只是在需要的时候临时构造一个匿名函数来使用 (例如需要使用函数作为参数进行输入时)。下面的例子展示了匿名函数的定义和调用方法:

```
1 >>> f = lambda x, y: x + y
2 >>> type(f)
3 <class 'function'>
4 >>> f(2, 3)
5 5
```

我们再分析下面的例子:

```
1 >>> def f(g, x):
2 ...     return g(x) * 2
3 ...
4 >>> result2 = f(lambda y: len(y) + 1, 'Python')
```

```

5 >>> print(result2)
6 14
7 >>> result3 = f(str, True)>>> print(result3)
8 TrueTrue

```

上述代码中，我们先定义了一个函数`f`，该函数有两个参数，第一个参数是另一个函数`g`，第二个参数是`x`，在运行函数`f`时，我们先调用函数`g`，并将`x`作为其参数传入，计算出`g(x)`的值后，再返回`g(x) * 2`。代码的第四行，我们定义了一个匿名函数`lambda y: len(y) + 1`作为函数`f`的第一个参数，所以，此时在函数`f`中，`g(x)`就是上述匿名函数，因此，此时`g(x)`的值就是`len('Python') + 1`，即 7，因此最终`f(g, x)`的输出为 14。代码的第七行，函数`f`的第一个参数是将变量转换为字符串的函数`str`，因此最终输出是`str(True) * 2`。

3.4.2 可选参数和可变数量参数

在 Python 中，我们可以将函数的某些参数定义为可选参数，实际上就是给这些参数设置默认值，即如果用户不指定该参数，则使用默认值进行计算，其定义的语法为：

```

1 def <函数名>(<参数列表>, <可选参数1>=<默认值1>, <可选参数2>=<默认值2>, ...):
2     <函数体>
3     return <返回值列表>

```

注意，在定义此类函数时，可选参数需要写在非可选参数的后面。下面的例子定义了一个包含可选参数的函数：

```

1 >>> def dup(string, times=2):
2     ...     print(string * times)
3     ...
4 >>> dup('RUC')
5 RUCRUC
6 >>> dup('RUC', 4)
7 RUCRUCRUCRUC

```

上面的例子中，函数`dup`会将输入的字符串参数重复`times`遍并输出，其中，`times`是可选参数，如果用户没有指定该参数（代码第 4 行），则使用默认值 2，而如果用户指定了该参数（代码第 6 行），则改为使用用户指定的参数。

我们还可以定义可变数量的参数，定义的方法是在最后增加一个参数，并以“*”开头，超出数量的参数最终都会被合并转换成一个元组，并由该以“*”开头的参数接受。举例如下：

```

1 >>> def calc_sum(a, *b):
2     ...     res = a
3     ...     for item in b:
4     ...         res += item
5     ...     return res
6     ...
7 >>> calc_sum(1, 2, 3, 4)
8 10

```

上面的例子中，实际调用函数`calc_sum`时传入了 4 个参数，其中，参数`a`对应第一个参数 1，而其他参数 2, 3, 4 被转换成一个元组(2, 3, 4)并传入第二个参数`b`。在函数`calc_sum`的定义中，我们使用`for`循环不断取出元组中的元素，并使用该元素与`res`的和来更新`res`，最终求得所有传入参数的和。

3.4.3 函数的返回值

在 Python 中，一个函数可以返回 1 个或者多个值，也可以不返回任何值。前文中我们已经介绍了返回 1 个值或不返回值的情况，如果函数需要返回多个值，我们可以将多个值用逗号隔开，此时 Python 解释器会将需要返回的多个值组成一个元组返回，因此本质上还是返回一个值。

```
1 >>> def f(a, b):
2 ...     return b, a
3 ...
4 >>> f(1, 2)
5 (2, 1)
```

通过上述例子，我们可以看出，函数返回的多个值会被组合成一个元组。如果想要将函数返回的多个值赋值给多个变量，我们可以将多个变量用逗号隔开的方式同时赋值：

```
1 >>> a, b = f(1, 2)
2 >>> a
3 2
4 >>> b
5 1
```

3.4.4 函数对变量的作用

在一个程序中，变量一般分为两类，全局变量和局部变量，全局变量在程序执行的全过程都有效，而局部变量一般存在生命周期，有生效和失效的时刻。我们在函数中定义的变量（包括形式参数），一般都是局部变量，它们只在调用函数时开始生效，而在函数执行完成后就失效。

```
1 >>> def f(num):
2 ...     result = num ** 2
3 ...     return result
4 ...
5 >>> square = f(3)
6 >>> print(result)
7 Traceback (most recent call last):
8   File "<stdin>", line 1, in <module>
9 NameError: name 'result' is not defined
```

上面的例子中，`result` 就是局部变量，当函数执行完成后，该变量就会失效，因此，尝试输出该变量的值就会报出未被定义的错误。

在函数的定义中，我们可以使用全局变量，例如：

```
1 >>> n = 2      # 全局变量
2 >>> def f(num):
3 ...     result = num ** n  # 使用全局变量
4 ...     return result
5 ...
6 >>> f(3)
7 9
```

注意，如果我们试图修改一个全局变量，Python 会创建一个同名的局部变量来进行操作，例如：

```
1 >>> n = 2      # 全局变量
2 >>> def f(num):
3 ...     n = num    # 此处的n是局部变量
4 ...
5 >>> f(1)
6 >>> print(n)
7 2
```

该例子中，函数中的变量 `n` 是一个局部变量，因此修改该变量并不会影响函数外定义的全局变量 `n`。如果确有必要修改全局变量，我们可以使用 `global` 保留字来显式说明：

```
1 >>> n = 2      # 全局变量
2 >>> def f(num):
3 ...     global n    # 显式声明使用全局变量
4 ...     n = num
5 ...
6 >>> f(1)
7 >>> print(n)
8 1
```

出现上面现象的原因在于，Python 解释器在寻找一个变量时，会首先在同级代码块（即拥有相同缩进量的代码块）中寻找，如果没有找到，那么 Python 解释器会继续在上一级代码块（即少一次缩进的代码块）中寻找，并以此类推。例如：

```
1 >>> n = 2      # 全局变量
2 >>> def f(num):
3 >>> def f(num):
4 ...     n = num
5 ...     def g(num2):
6 ...         print(num2 * n)
7 ...     g(5)
8 ...
9 >>> f(3)
10 15
```

上面的例子中，我们在函数 `f` 中定义了另一个函数 `g`，在函数 `g` 中，我们使用了变量 `n`，同级代码块中并没有该变量，因此，Python 解释器中会在上级代码块，即函数 `f` 中寻找，函数 `f` 中定义了一个局部变量 `n`，因此，在执行 `g(5)` 时，Python 解释器会采用代码第 3 行定义的变量 `n`。

而 `global` 保留字改变了 Python 解释器的行为，改为直接寻找相应的全局变量，例如：

```
1 >>> n = 2      # 全局变量
2 >>> def f(num):
3 >>> def f(num):
4 ...     n = num
5 ...     def g(num2):
6 ...         global n    # 声明使用全局变量n
7 ...         print(num2 * n)
```

```

8 ...     g(5)
9 ...>>> f(3)
10 10

```

与之前的例子相比，我们增加了一行代码，声明使用全局变量 `n`，此时 `g()` 函数在执行时会直接找到最外层的全局变量 `n`，因此最终的输出变为 10。



使用全局变量是一个比较危险的动作，可能会造成意想不到的结果，因此，在任何情况下，我们都不建议使用全局变量。如果需要在不同代码或函数之间共享数据，我们可以将需要共享的数据作为函数的参数和返回值进行传递。

3.4.5 函数的递归

函数可以封装一段代码，被其他程序或代码调用，当然也可以被自己调用，在一个函数的定义中调用自身的方式称为递归。递归在数学和计算机应用上非常强大，可以非常简洁地解决一些问题。在数学上，一个经典的例子是阶乘的定义，自然数 n 的阶乘使用 $n!$ 来表示，其定义为：

$$n! = \begin{cases} 1 & n = 0 \\ n \cdot (n-1)! & \text{其他情况} \end{cases} \quad (3.1)$$

可以看到，递归定义一般满足以下两个特征：

- 存在一个或多个基例，基例不需要通过递归定义，它们有确定的表达式；
- 所有的递归链要以基例结尾。

如果使用 Python 编程计算阶乘，我们可以编写如下的代码：

```

1 def fact(n):
2     if n == 0:
3         return 1
4     else:
5         return n * fact(n - 1)

```

可以看到，上面的代码就是公式(3.1)的简单重新描述。以计算 3 的阶乘为例，图3.3展示了函数的调用关系。

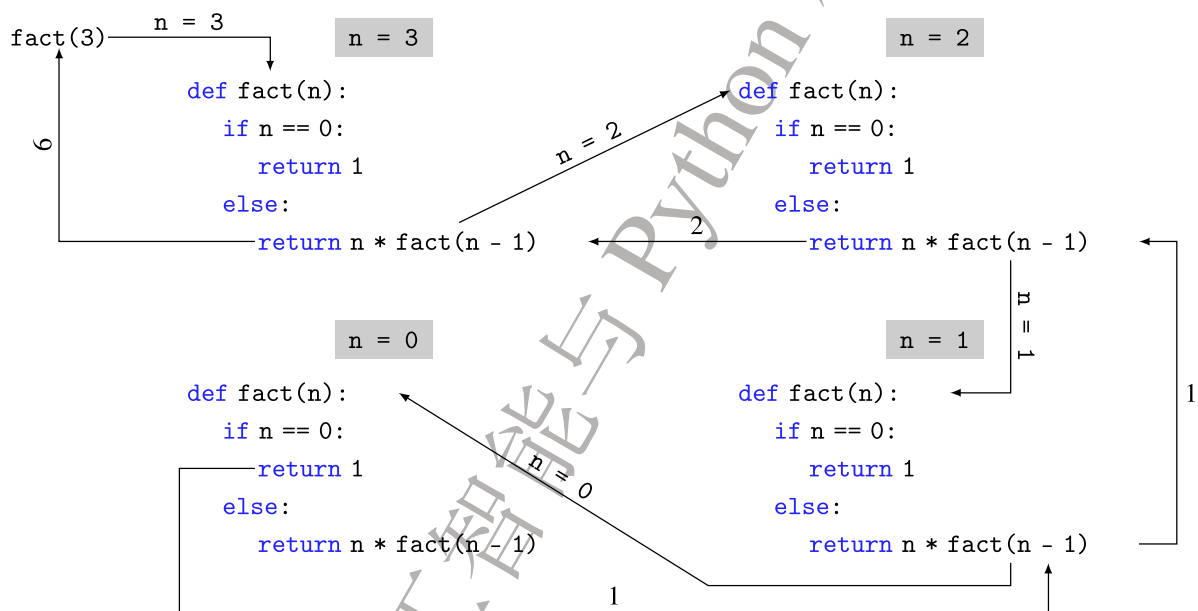
下面的函数使用递归的方法将一个字符串反转：

```

1 def reverse(s):
2     if s == '':
3         return s
4     else:
5         return s[1:] + s[0]

```

当我们需要反转一个长度为 n 的字符串时，我们可以这样考虑：假设我们可以反转一个长度为 $n-1$ 字符串，那么我们只需要将原来字符串的后 $n-1$ 个字符构成的子串进行反转，再将原字符串的第一个字符放置在最后即可。当然我们还需要一个基例，即如何反转长度为 0 的字符串，该基例的答案是显然的。

图 3.3: 计算`fact(3)`时的函数递归调用图